OSPSD HW2 Rubric - Spring '26

The assignment is out of **100 points** but there are **110 points** you can get (so you have some leniency).

1. Repository & Process (15 pts)

[5 pts] PR submitted correctly: `hw-2` branch → `main`, not merged. PR has descriptive title and clear summary.

[5 pts] Repository structure: Correct `src/` layout; all installable packages (including the auto-generated client) are workspace members. No stray top-level modules. `.gitignore` covers `.venv`, `__pycache__`, `*.pyc`, credentials (`credentials.json`, `token.json`), `.env`, and build artifacts. GitHub templates (PR + Issue) from HW1 are maintained.

[5 pts] Commit quality: Clean, imperative messages; logical, small commits following the squash-merge workflow discussed in lecture.

2. Tooling & Configuration (15 pts)

[5 pts] `uv` workspace & centralized configs: Root `pyproject.toml` is a workspace with all `src/*` members listed (including the auto-generated client). `ruff`, `mypy`, `pytest`, `coverage` settings live in root `pyproject.toml` only. No `requirements.txt` or `pip`.

[5 pts] Analyzer strictness & import hygiene: `ruff` uses `select = ["ALL"]` with justified `ignore` entries. `mypy` uses `strict = true` with narrow, documented exceptions. No blanket `type: ignore` or `noqa` without justification. Only absolute imports used throughout.

[5 pts] Deps hygiene: Dev deps under `[project.optional-dependencies]`; runtime deps minimal per package. Each component declares only its own runtime deps; component-specific dependencies are not hoisted into the root `pyproject.toml`.

3. New Components (20 pts)

You should've built the three bridges: the FastAPI service (wrapper), the auto-generated client (mechanical HTTP layer), and the adapter (shim back to your original interface).

[3 pts] Clear componentization: Components are properly split (`[vertical]_api`, `[vertical]_impl`, `[vertical]_service`, `[vertical]_service_api_client`, `[vertical]_adapter`). Each component has a single responsibility. The existing `_api` package remains framework-free; HW2 work does not introduce new dependencies into it.

[7 pts] FastAPI service: `[vertical]_service` wraps the concrete implementation over HTTP.

- The service imports and delegates to the existing impl. It does not reimplement business logic that already exists in the impl.
- Endpoints cover the core functions defined in the abstract API.
- OAuth 2.0 Authorization Code Flow endpoints are present (`/auth/login` , `/auth/callback`): the service redirects to the provider, handles the callback, exchanges the authorization code for tokens, and stores credentials in a session.
- `/health` endpoint returns HTTP 200.
- Domain exceptions from the impl are translated into appropriate HTTP status codes (not raw 500s).

[4 pts] Auto-generated client: `[vertical]_service_api_client` exists as a workspace member and was generated from the service's `/openapi.json` using `openapi-python-client` . The adapter imports from this package. The adapter does not hand-roll HTTP calls via `requests` or `httpx` directly.

[6 pts] Service client adapter: `[vertical]_adapter` wraps the auto-generated client and implements the original abstract API.

- The adapter maps the generated client's types and methods onto the original interface. It does not contain business logic.
- HTTP errors from the generated client are translated into domain exceptions defined in the `_api` package.
- Swapping between the local impl and the remote adapter is transparent to the consumer: importing either one registers it via `get_client()` (or equivalent), and the consumer's code does not change. This is the adapter pattern sanity check described in the assignment.

4. Domain Modeling & API Design (8 pts)

[2 pts] DTO separation & framework-agnostic domain: Business actions belong on the high-level ports, not on data classes. Use built-ins (`dataclass` or `ABC`) in domain/ports; Pydantic only at edges (HTTP service and adapters) for validation/serialization.

[2 pts] No leakage: External concerns (auth tokens, HTTP clients, SDK types) do not leak into domain/port packages. OAuth token management lives in the service and adapter layers, not in the `_api` package.

[4 pts] Error modeling: Ports raise typed domain exceptions instead of returning `None` for control flow. Errors are translated at both adapter boundaries: the adapter converts HTTP errors

into domain exceptions, and the service converts domain exceptions into HTTP status codes.

5. Testing Strategy (18 pts)

[7 pts] Unit tests (per component): Live under each component's `tests/`. Use `unittest.mock`/fakes for external services (no network calls). Tests should exercise the public API, assert on state and output (not method invocation), cover each distinct behavior, and use hardcoded expected values rather than recomputing logic from the source. Verify ports raise proper domain exceptions.

[5 pts] Integration tests (`tests/integration/`): Validate DI wiring (the abstract factory returns the correct concrete type after import). Exercise a real adapter path against a sandbox or recorded responses, asserting domain mapping and error translation. Integration tests that mock all components are an anti-pattern.

[6 pts] E2E tests (`tests/e2e/`) & coverage: Run the app entry point as a subprocess with black-box assertions on user-visible behavior. This is also the natural place to demonstrate location transparency (same consumer code, both backends). Coverage threshold defined in `pyproject.toml`; CI build fails if coverage drops below it.

6. CI/CD & Deployment (10 pts)

[4 pts] CircleCI pipeline: `.circleci/config.yml` runs lint (`ruff`), types (`mypy`), and all tests (unit, integration, E2E). CI is passing on `hw-2` and is publicly visible. Coverage reports uploaded and browsable. Test results accessible in CircleCI's Tests dashboard.

[3 pts] Service deployed to a public cloud. Reachable base URL serves `/openapi.json` and `/health`.

[3 pts] Automatic deployment: CircleCI triggers a build and deploy on push to `hw-2`. Secrets managed via platform's native secrets manager (not committed to source control).

7. Documentation (7 pts)

[2 pts] Root `README.md`: Purpose, architecture overview (ports/adapters philosophy), setup/auth instructions, deployment steps (platform, URL, env vars), how to run the toolchain.

[2 pts] Component READMEs: Each `src/*` package documents its API, dependencies, and role.

[3 pts] MkDocs site: `mkdocs.yml` present; `docs/` directory populated and up to date; documentation builds without errors. HW2 content is reflected in the navigation.

8. Peer Review (10 pts)

[5 pts] Review given: Team left constructive, substantive comments on assigned team's PR that show genuine engagement with their code and design.

[5 pts] Review addressed: Team responded to the peer review feedback received on their PR, either implementing the suggestion or providing a written rationale for declining. Feedback addressing is mandatory, not optional.

9. DESIGN.md (7 pts)

[7 pts] Design document covers the required sections (architecture overview, request flow, API design with error handling, adapter pattern rationale with code comparison, and testing strategy). Thorough enough for an outside contributor to understand the system and the design decisions behind the new components.

Extra Credit (+10 pts)

Up to +10 points for going above and beyond. Examples:

- Containerized deployment (Docker) with a working `Dockerfile` and documented build/run instructions.
- Multi-user session management: the service correctly isolates OAuth tokens per user rather than sharing a single credential.

First Draft

The first draft deadline is for feedback, not grading. However, teams that submit nothing or an empty PR by the first draft deadline forfeit the iterative feedback cycle, which historically correlates with lower final scores. The final submission is what gets graded.